

Cricket Guru

A cricket question-answering agent grounded in structured ball-by-ball match data and encyclopedic prose, so it answers both "who scored the most in IPL 2016" and "why the 2019 final was controversial" without making either up.

Group: IP8, Cricket Guru | **Primary framework:** Pydantic AI | **Domain:** Cricket — International men's (Test / ODI / T20I) + IPL

PROBLEM STATEMENT

Cricket knowledge lives in two incompatible shapes, and no single retrieval method serves both. **Prose** holds narrative and judgement: why a match mattered, how a technique worked, what made a decision contentious. **Structured records** hold exact facts: run tallies, strike rates, results. Ask a vector search for the most runs in a season and it returns a paragraph that sounds right and is often wrong; ask a stats database why a chase was famous and it has no column for that. A third failure sits on top: **records go stale** the moment they break, so a retrieval-only system answers record questions confidently and out of date.

Cricket Guru reads each question, decides which source actually holds the answer, and pulls from it: exact numbers from a structured match DB, narrative from a text index, and a web-search freshness check for records. The scope for now: men's international cricket and the IPL, factual and narrative questions with checkable answers.

The kinds of questions it must separate:

- *Fact*: "Who scored the most runs in IPL 2016?" → structured match data.
- *Narrative*: "Why was the 2019 World Cup final decided on boundary count?" → prose.
- *Record / stale*: "Who holds the record for most sixes in men's T20Is?" → structured data + web freshness check, not a years-old article.

DATA SOURCES & TOOLS

Source	What it is (ingested, verified)	Store	Tool it powers
Cricsheet	Open ball-by-ball, men's intl + IPL. 8,142 matches, 4.14M deliveries.	PostgreSQL (matches, innings, deliveries, players)	Stats query — agent writes SQL to compute exact numbers
Wikipedia (cricket)	575 encyclopedic articles, 8.3M chars. The narrative source.	Qdrant vector DB (bge-small via FastEmbed)	Semantic search — relevant prose passages, with citations
Sports Stack Exchange	Cricket-tag Q&A. 839 Q, 459 accepted answers.	PostgreSQL (never embedded)	Evaluation oracle — accepted answers as the narrative answer key

Agent tools (Pydantic AI ReAct loop, Claude Sonnet): stats-query (SQL over Cricsheet) · semantic-search (Qdrant) · **web-search freshness**. **Grounding policy:** corpus-first; when retrieval is insufficient the agent falls back to web-search instead of fabricating, and labels what came from corpus vs web.

COMPARATIVE EVALUATION

Ground truth is found and computed, not hand-authored, on two tracks: **stats** questions templated from Cricsheet with **SQL-computed** answers (exact match); **narrative** questions from Sports SE accepted answers, scored by **LLM-as-judge** and validated against human labels.

We do not just reach for the optimal design. At each pipeline leg we compare **two prominent approaches**, varying one leg at a time against a simple baseline, so each result shows what the advanced technique actually buys:

- **Chunking:** fixed-size vs structural
- **Retrieval:** dense vs hybrid (dense+BM25)
- **Routing:** rule/keyword router vs LLM tool-calling agent
- **Judge:** same-model vs cross-model (self-preference bias)

Reported per variant: accuracy delta, plus hallucination/groundedness, routing accuracy, cost/latency, and web-fallback rate. Per-question logs feed the failure analysis.

INTERFACE

A Streamlit app with two views: a **chat** (answer + tool trace + sources + corpus-vs-web transparency) and a **comparisons dashboard** that shows the leg-by-leg results and metrics intuitively.